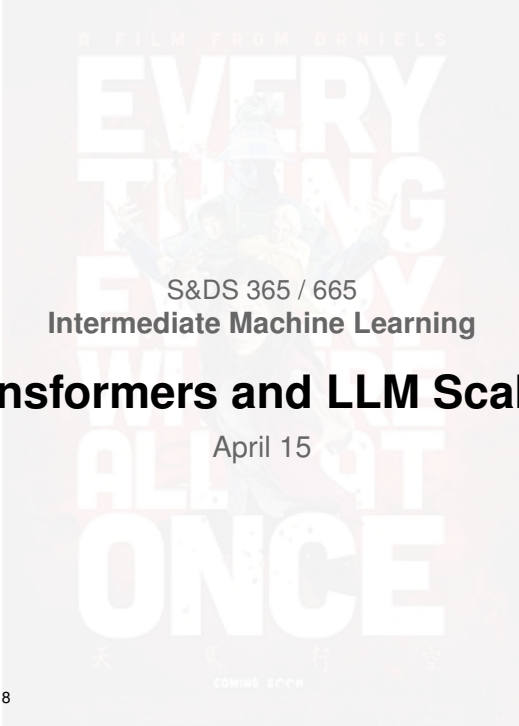




A FILM FROM DANIELS
**EVERY
THING
EVERYWHERE
ALL
AT
ONCE**
天 馬 行 空
COMING SOON

S&DS 365 / 665
Intermediate Machine Learning

Transformers and LLM Scaling

April 15

Yale

For Today

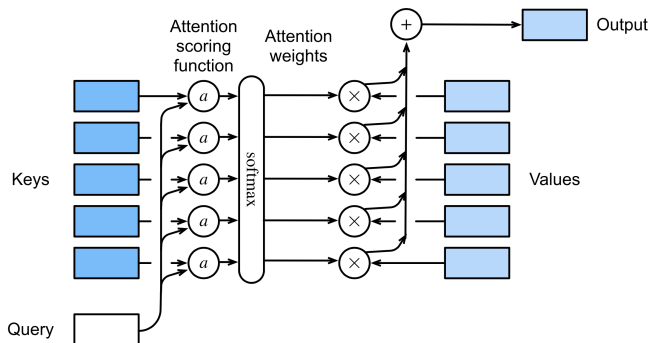
Sequence models

- Transformer architecture
- Pseudocode
- LLM scaling laws
- Finetuning

Attention, please!

- Quiz 5 (last!) is posted (RL, HMMs, RNNs, GRUs), live after class today
- Assn 5 out; due next Thursday (GRUs, Transformers)
- Final exam: Tuesday, May 5, 2pm

Attention in terms of query/key/values



Attention in terms of query/key/values

$$\begin{aligned}\text{Attn}(q, \{(k_1, v_1), \dots (k_m, v_m)\}) &= \text{Attn}(q, (k_{1:m}, v_{1:m})) \\ &= \sum_{i=1}^m w_i(q, k_{1:m}) v_i\end{aligned}$$

where weights w_i are softmax of attention scores $a(q, k)$:

$$w_i(q, k_{1:m}) = \frac{\exp(a(q, k_i))}{\sum_{j=1}^m \exp(a(q, k_j))}$$

Attention



On each step of decoding, directly connect to the encoder, and focus on a particular part of the source sequence

Attention



On each step of decoding, directly connect to the encoder, and focus on a particular part of the source sequence

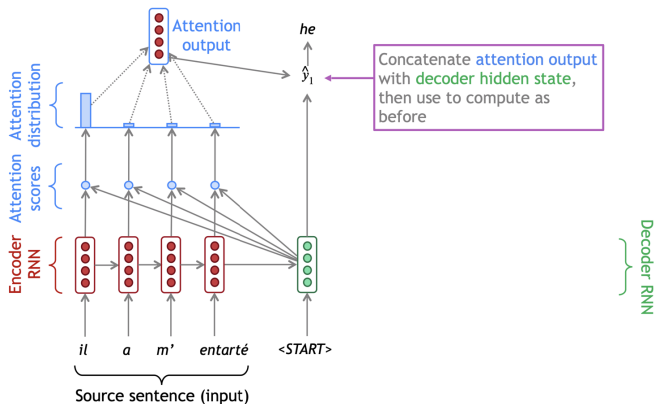


figure source: <http://web.stanford.edu/class/cs224n/slides/cs224n-2020-lecture08-nmt.pdf>

Attention



On each step of decoding, directly connect to the encoder, and focus on a particular part of the source sequence

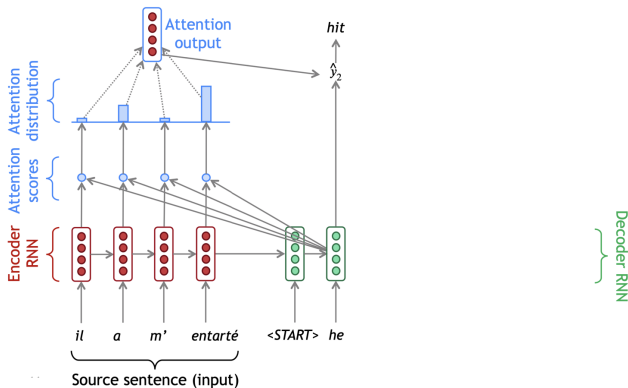


figure source: <http://web.stanford.edu/class/cs224n/slides/cs224n-2020-lecture08-nmt.pdf>

Attention



On each step of decoding, directly connect to the encoder, and focus on a particular part of the source sequence

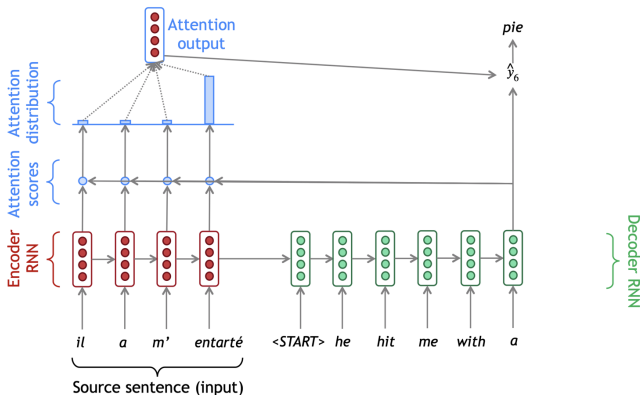
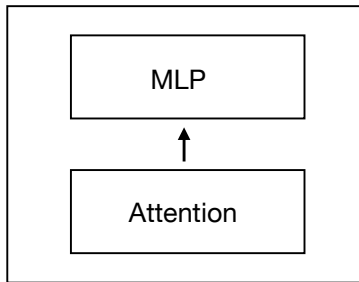


figure source: <http://web.stanford.edu/class/cs224n/slides/cs224n-2020-lecture08-nmt.pdf>

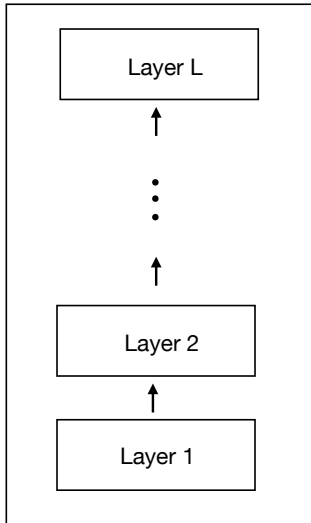
Key steps in Transformers

- *Tokenize* the text into a fixed vocabulary
- *Embed* the tokens into high-dimensional vectors
- *Mix* the embeddings in the context using “Attention”
- *Map* the resulting vectors using a neural network
- *Normalize* to keep size from growing
- *Add* to the residual stream

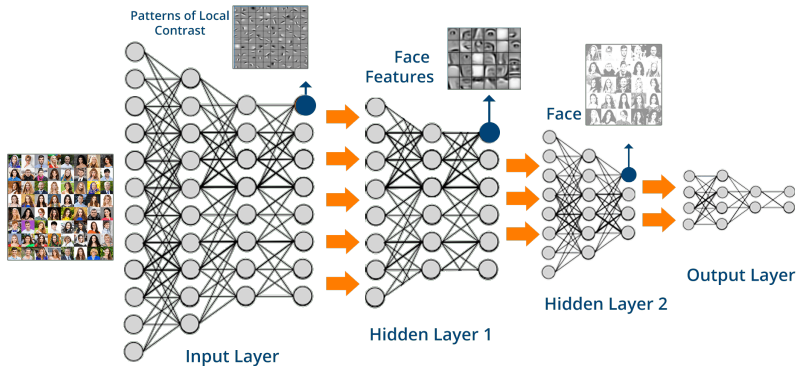
Transformer Layer



Transformer

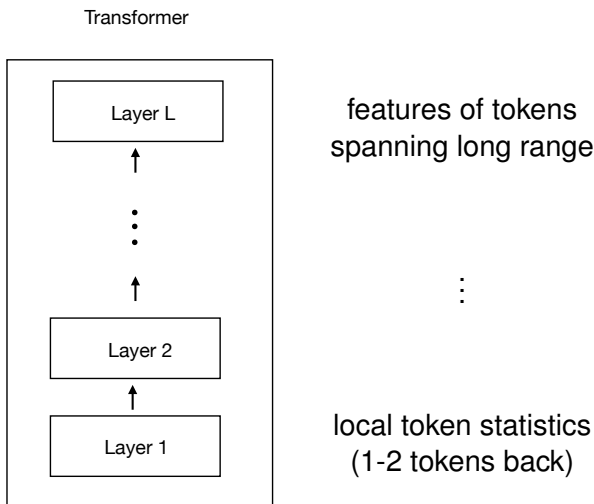


Increasing complexity of attention patterns



- Early layers capture local patterns of image contrast
- Later layers capture larger patterns that match parts of faces

Increasing complexity of attention patterns



Positional encoding

Positional encoding is an interesting way of getting position information into the model

Positional encoding

- ▶ Positions are integers, but neural nets can't handle integers
- ▶ Could use binary encoding; fixed precision
- ▶ Transformers use a sine/cosine basis

Positional encoding

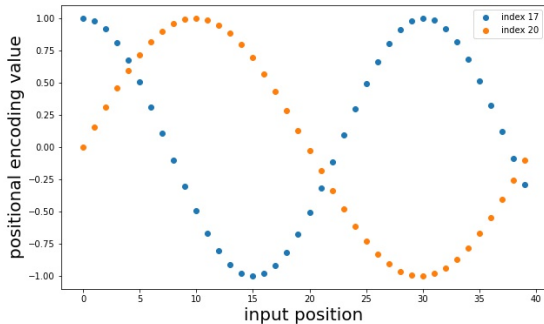
For the t th word in the sequence, components are

$$\text{PE}_{(t,2i)} = \sin\left(\frac{t}{C^{2i/d}}\right)$$

$$\text{PE}_{(t,2i+1)} = \cos\left(\frac{t}{C^{2i/d}}\right)$$

where d is the embedding dimension and C is a maximum sequence length

Positional encoding

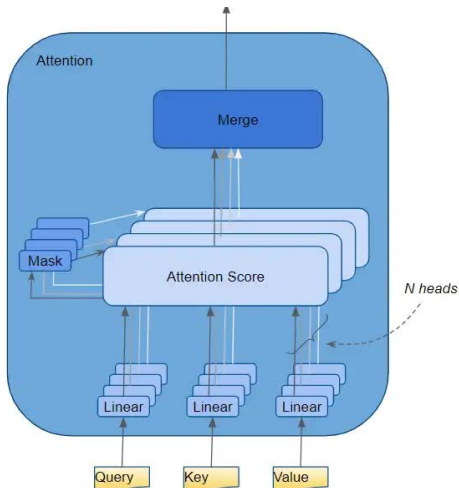


Two components for a length 40 input

Positional encoding

- Encoding of one position is linearly predictable from another, given their offset distance (phase shift)
- If the positional encoding is removed, the resulting model is permutation equivariant

Multihead Attention



Multiple attention vectors are computed, then merged (concatenated)

Transformer architecture: Pseudocode

```
def Transformer(tokens):  
    x = embed_tokens(tokens)  
    x = x + positional_encoding(len(tokens))  
  
    for l in [L]:  
        normed_x = layer_norm(l, x)  
        attn_out = Attention(l, normed_x)  
        x = x + attn_out  
        normed_x = layer_norm(l, x)  
        mlp_out = mlp(l, normed_x)  
        x = x + mlp_out  
  
    logits = x @ W + b  
    return logits
```

Attention: Pseudocode

```
def Attention(l, x)
    head_outputs = []
    for h in [n_heads]:

        Q = x @ W_q[l, h]
        K = x @ W_k[l, h]
        V = x @ W_v[l, h]

        attn_scores = Q @ K.T / sqrt(d)
        attn_scores = Softmax(attn_scores)

        head_out = attn_scores @ V
        head_outputs = head_outputs + head_out

    return head_outputs @ W_o[l]
```

MLP: Pseudocode

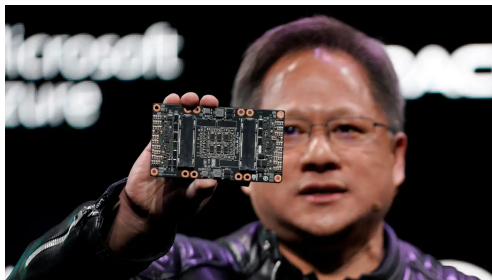
```
def MLP(l, x)

    h = x @ W1[l] + b1[l]
    h = relu(h)
    o = h @ W2[l] + b2[l]

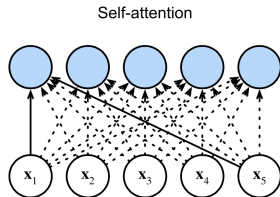
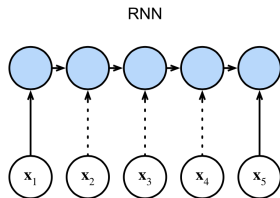
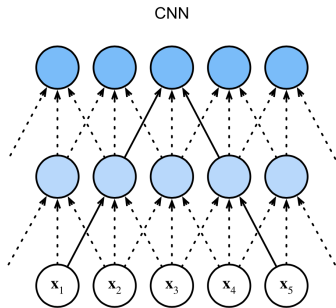
    return o
```

Computation

- Everything is implemented as big matrix multiplies
- Carried out very efficiently on GPUs
- GPUs are optimized for linear equations (vector graphics)
- Allows scaling models to very large size



Conceptual comparison: CNN, RNN, Transformer



Conceptual comparison: CNN, RNN, Transformer



Final step

Generating new tokens

Rolling a D50,257



- ▶ Transformer assigns a score to each token
- ▶ Converted to a probability
- ▶ Generating next token equivalent to rolling weighted 50,257-sided die

Generation

Generation algorithm has two “knobs” to control:

- 1 *Top-k*: Restricts to just the top k tokens
 - ▶ Low k : Low variability
 - ▶ High k : Higher variability

Generation

Generation algorithm has two “knobs” to control:

- ② *Temperature T* : Lower temperature favors more likely tokens
 - ▶ Low temperature: Low variability
 - ▶ High temperature: High variability

Generation

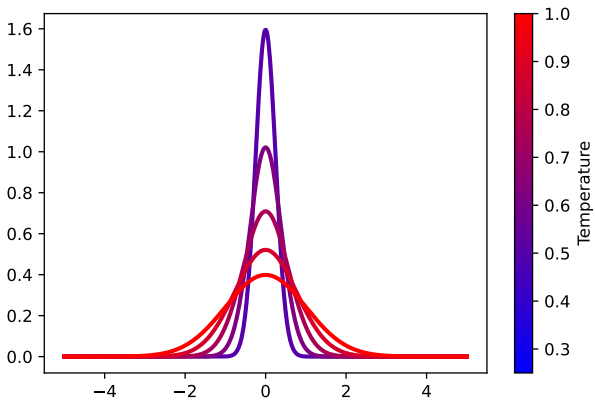
At temperature T , if the scores of the top k tokens are

$$\text{score}_1, \text{score}_2, \dots, \text{score}_k$$

the weights on the die are proportional to

$$e^{\text{score}_1/T}, e^{\text{score}_2/T}, \dots, e^{\text{score}_k/T}$$

Temperature



Annealing



Wikipedia: Annealing (materials science)

Temperature $T = 1$

One day, Timmy met a monster. The monster was big and green, but it was harmless. Timmy was scared, but he wanted to be brave. He walked closer and closer to the monster. Suddenly, the monster snapped its fingers! Timmy was so scared that he started to cry. The monster said, "Don't be scared! I'm harmless!" But Timmy was still sc

One day, Timmy met a monster. The monster had big teeth and green skin. Timmy was scared and ran away. But the monster called out, "Wait! I don't want to hurt you. I just want to be your friend." Timmy stopped and looked back. "Really?" he asked. "Yes, I permit you to be my friend," the monster replied. Timmy was surprised. He didn't expect

Temperature $T = 0$

One day, Timmy met a monster. The monster was big and scary. Timmy was scared and ran away. The monster chased Timmy. Timmy ran and ran until he found a big tree. He climbed up the tree and the monster couldn't reach him. Timmy was safe. Timmy saw a bird and said, "Hello bird, can you help me? The monster is chasing me." The bird said, "

One day, Timmy met a monster. The monster was big and scary. Timmy was scared and ran away. The monster chased Timmy. Timmy ran and ran until he found a big tree. He climbed up the tree and the monster couldn't reach him. Timmy was safe. Timmy saw a bird and said, "Hello bird, can you help me? The monster is chasing me." The bird said, "

Temperature $T = 1.5$

One day, Timmy met a monster. The monster was big and scary. Timmy didn't know what to do. He felt bitter because he wanted to play with the monster. The monster said, "Hello, Timmy. Can you show me around the forest?" Timmy wasn't sure if he believed him or not. He said, "I'm sorry, do you have room for crawlers?" The monster guesses sad

One day, Timmy met a monster. The monster was big and scary, but Timmy wasn't scared. "Hi," said Timmy. The monster was surprised. He couldn't understand what Benny had said. "What are you up to?" he asked. Timmy picked him up one by one and he saw doziles of Ell Party coming in on a streak street. "I amdri catientased!"

Generation: Pseudocode

```
def generate(prompt_text, temperature):  
    generated_tokens = tok.encode(prompt_text)  
  
    for i in [tokens_to_generate]:  
        logits = Transformer(generated_tokens)  
        if temperature == 0:  
            next_token = argmax(logits)  
        else:  
            logits = logits / temperature  
            probs = softmax(logits)  
            next_token = sample(vocab_size, probs)  
        generated_tokens += next_token  
  
    generated_text = tok.decode(generated_tokens)  
    return generated_text
```

The Transformer architecture

How large are the models?

How does size of LLM scale?

Design choices: number of tokens V , dimension d of embeddings, and number of Transformer layers L

- Embeddings have $d \cdot V$ parameters
 - ▶ d numbers for each token, V tokens

How does size of LLM scale?

Design choices: number of tokens V , dimension d of embeddings, and number of Transformer layers L

- Self-Attention has $\approx 4d^2$ parameters
 - ▶ $d \times d$ matrix for queries (across heads)
 - ▶ $d \times d$ matrix for keys (across heads)
 - ▶ $d \times d$ matrix for values (across heads)
 - ▶ $d \times d$ matrix for combining heads

How does size of LLM scale?

Design choices: number of tokens V , dimension d of embeddings, and number of Transformer layers L

- MLP applied after has $\approx 8d^2$ parameters
 - ▶ Two layers
 - ▶ Hidden layer has $4 \cdot d$ neurons
 - ▶ Output layer has d neurons
 - ▶ $4d^2$ weights in each layer; $8d^2$ weights overall

How does size of LLM scale?

Design choices: number of tokens V , dimension d of embeddings, and number of Transformer layers L

- Embeddings have $d \cdot V$ parameters
- In each Transformer layer:
 - ▶ Attention has about $4d^2$ parameters
 - ▶ MLP has about $8d^2$ parameters
- Total trainable parameters: $12d^2 \cdot L + d \cdot V$

Size of models

GPT-2 (2019)

$$d = 1,600$$

$$V = 50,257$$

$$L = 48$$

$$12 \cdot d^2 \cdot L + d \cdot V \approx 1.5 \text{ billion parameters}$$

- ▶ Each GPT-2 Transformer layer has about 30 million parameters

Size of models

GPT-3 training data^{[1]:9}

Dataset	# tokens	Proportion within training
Common Crawl	410 billion	60%
WebText2	19 billion	22%
Books1	12 billion	8%
Books2	55 billion	8%
Wikipedia	3 billion	3%

Size of models

GPT-3 (2020)

$$d = 12,288$$

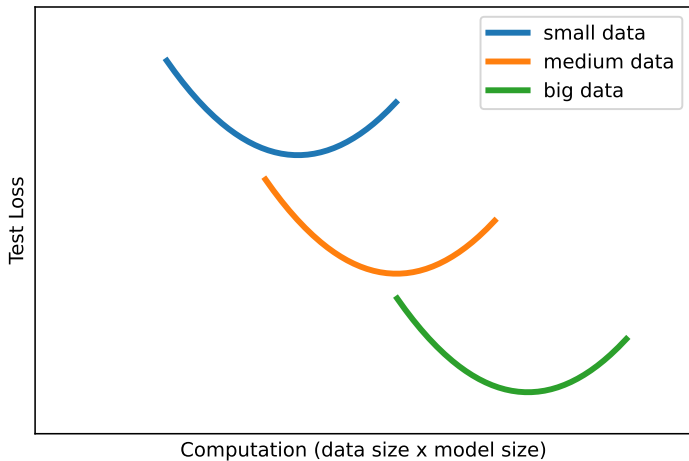
$$V = 50,257$$

$$L = 96$$

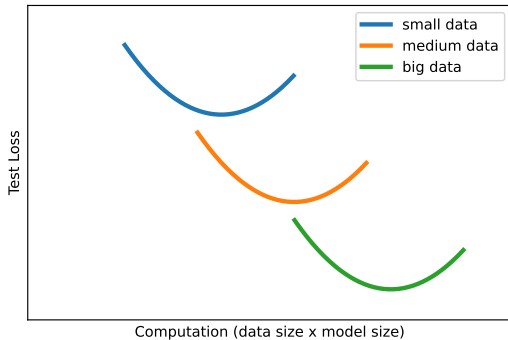
$$12 \cdot d^2 \cdot L + d \cdot V \approx 173 \text{ billion parameters}$$

- ▶ Each GPT-3 Transformer layer has about 1.8 billion parameters

Scaling behavior of LLM models



Scaling behavior of LLM models





Original painting by Johannes Vermeer; transformed (pixelated) by acagastyaa, CC0, via Wikimedia Commons

ANNALS OF ARTIFICIAL INTELLIGENCE

CHATGPT IS A BLURRY JPEG OF THE WEB

OpenAI's chatbot offers paraphrases, whereas Google offers quotes. Which do we prefer?

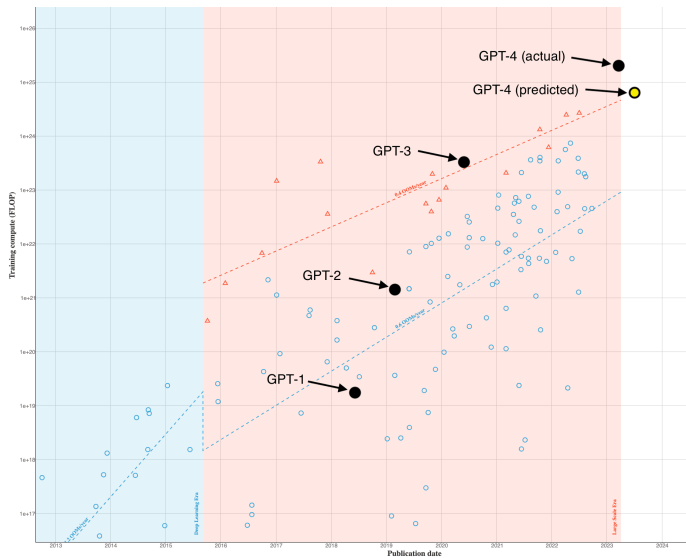
By Ted Chiang

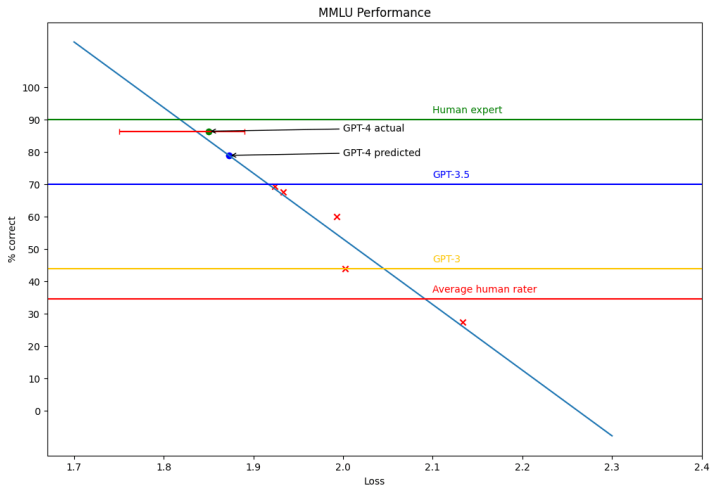
February 9, 2023

Back of the envelope calculation



- LLM loss on test data is ≈ 1.9 nats/token
- Translates to entropy of about $1.9 / \log_e(2) \approx 2.74$ bits/token
- So, *lossless* coding of 500 billion tokens takes ≈ 171 gigabytes
- GPT-3's 175 billion params uses 346 gigabytes (16-bit precision)
- GPT-3 could, in principle, memorize most of the internet!





MMLU: Massive Multitask Language Understanding, <https://en.wikipedia.org/wiki/MMLU>,
<https://paperswithcode.com/dataset/mmlu>

Sutton's "Bitter Lesson" (2019)



“The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin.”